

Recap Checkpoint 5 — Cumulative: Component 1 (1.1, 1.3, 1.4) + Component 2 (2.1–2.3, incl. Algorithms)

Name: _____ Date: _ **Mark:** _____ / 35

OCR H446 cumulative recap — mixes every topic taught so far. Show all working.

Questions

Q1 [3] (AO1). (1.1 — Architecture) Compare **Von Neumann** and **Harvard** architectures by stating **one** difference in how they store instructions and data, and giving **one** advantage of each.

Q2 [4] (AO2). (1.4 — Calculations) Show all working.

(a) Convert hexadecimal **3B** to denary. **[2]**

(b) A bitmap image is 64×48 pixels with a colour depth of **4 bits per pixel**. Calculate its size in **kibibytes (KiB)**, ignoring metadata. **[2]**

Q3 [4] (AO2). (1.4 — Boolean algebra)

.....

.....

.....

.....

.....

.....

(a) Simplify the Boolean expression $A \cdot B + A \cdot \bar{B}$ and name the law(s) used. [2]

.....

.....

.....

(b) Complete the output column of the truth table for $Q = (A \text{ OR } B) \text{ AND NOT } B$. [2]

.....

A	B	Q
0	0	
0	1	
1	0	
1	1	

Q4 [3] (AO1). (1.3 — Databases) A relational database has tables `Student` and `Course` .

.....

.....

.....

.....

(a) State what is meant by a **foreign key**. [1]

.....

.....

(b) Explain how **normalisation to 3NF** reduces data redundancy. [2]

Q5 [4] (AO2/AO1). (2.2 — *Programming paradigms*)

(a) State **one** key difference between **procedural** and **object-oriented** programming. [1]

(b) Define **encapsulation** and explain **one** benefit it provides. [2]

(c) State what is meant by **inheritance**. [1]

Q6 [4] (AO2). (2.3 — *Searching/sorting trace*) A list contains [8, 3, 5, 1]. Carry out a **merge sort**, showing the lists produced at **each** split and merge stage. State the **worst-case** time complexity of merge sort.

.....

.....

Q7 [4] (AO1/AO2). (2.3 — *Big O*)

.....

.....

.....

.....

.....

.....

(a) State the worst-case time complexity (Big O) of: linear search, binary search, and bubble sort. **[3]**

.....

.....

.....

(b) An algorithm contains a single loop that runs n times, and inside it a binary search over the same n items. State its overall time complexity. **[1]**

.....

Q8 [4] (AO2). (2.3 — *Graphs/trees*) For the binary search tree below, give the **in-order** traversal and state what property it demonstrates, then state the **order** in which 27 would be compared against existing nodes if inserted.

.....

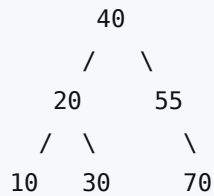
.....

.....

.....

.....

.....



Q9 [5] (AO3). (*Synoptic extended response — 2.1/2.2/2.3*) A music-streaming service must let users search a library of millions of songs by title and play results quickly.

Explain why the library should be **kept sorted** and a **binary search** used rather than a linear search, referencing **time complexity**, and identify **one** trade-off introduced by keeping the data sorted. **[5]**
